Enhancing Financial Fraud Detection through Identity-Aware Feature Engineering on the IEEE-CIS Dataset

Rameez Shafat Wani
School of Computing
Dublin City University
Dublin, Ireland
rameezshafat.wani2@dcu.mail.ie

Aditya Dhayagude
School of Computing
Dublin City University
Dublin, Ireland
adityatanaji.dhayagude2@mail.dcu.ie

David Ajayi
School of Computing
Dublin City University
Dublin, Ireland
david.ajayi3@mail.dcu.ie

Abbas Zaidi
School of Computing
Dublin City University
Dublin, Ireland
mehdiabbas.zaidi2@mail.dcu.ie

***Abstract*—Fraud detection is an extremely unbalanced binary classification task, where transactional data columns contain information about independent events without much predictive value. Context-aware and identity-specific features, obtained from cardholder data, spending history, and frequency of entities, are the main contributor to high-quality predictions, while the vast majority of literature about fraud detection focuses on model selection. The current paper describes a systematic and group-by-group approach to feature engineering in the IEEE-CIS Kaggle competition, using the CRISP-DM methodology (590,540 transactions, 3.5 % of fraud). Eight feature groups, taking into account their dependencies and avoiding leakage, are generated: a UID proxy, frequency encoding, cross-key combination frequencies, multi-granular amount aggregations, cross-key statistics, C-column behaviour counts, UID-level statistics, and out-of-fold (OOF) UID-level target encoding for fixing a label leakage problem. XGBoost, LightGBM and CatBoost are used for model generation. The full feature engineered list achieves a test ROC-AUC of 0.9470 after UID postprocessing; the consensus of feature importances proves that the most important features are obtained via UID and aggregation methods. The key finding of the research is that *not more complex model, but better feature engineering provides the most valuable performance lever*.**

***Index Terms*—fraud detection, feature engineering, identity proxy, behavioural aggregation, frequency encoding, target encoding, class imbalance, CRISP-DM, IEEE-CIS, gradient boosting**

I. Introduction

Online transaction fraud cost the financial industry an annual $32 billion, expanding at a rate comparable to that of e-commerce [1]. Automated fraud scoring is faced with the problem of classifying millions of transactions in the presence of very high class imbalance (∼3.5 % fraud), time-variant dynamics, and lack of a trustworthy customer identifier. The prevailing trend in current research is to compare models of ever-increasing complexity on such data sets [11], yet it is common knowledge in the field that a carefully crafted feature set will beat an advanced model using a poorly engineered data set. *The key issue addressed in this work is not model choice, but feature engineering.*

The following research questions will be addressed in this study:

- **RQ1:** What is the importance of which feature engineering classes to predict fraudulent transactions on the IEEE-CIS dataset?
- **RQ2:** Can identity-based feature extraction, especially UID and its aggregation functions, significantly improve fraud detection performance compared to plain transactional features?

CRISP-DM [2] is employed for the IEEE-CIS Vesta Corporation dataset. We develop eight groups of features in an ordered fashion, enabling the conversion of individual transactions into client behavior profiles. Three gradient boosting models are applied strictly as means of feature assessment only. The contributions include:

- Dependency-ordered taxonomy of eight feature engineering groups, based on distinct hypotheses regarding the fraud-behavior relation.
- Chronological out-of-fold target encoding approach, which rectifies the issue of label leakage associated with the v1 pipeline.
- Experimental confirmation of the superior importance of engineered features relative to raw ones when cross-model consensus importance ranks are considered.
- Bootstrapped hypothesis test confirming that the AUC improvement is statistically significant and attributable solely to the engineered feature groups.

II. Related Work

A. Model-Centric Fraud Detection

In almost all cases where performance measures have been considered for fraud, this performance is presented as a function of model choice. For imbalanced fraud problems using logistic regression and under-sampling, PR-AUC was found to be the appropriate measure by Dal Pozzolo et al. [3], showing that accuracy becomes inappropriate when considering low fraud rates. In a paper showing advantages of Random Forest over logistic regression in terms of implicit feature interactions, Sahin and Duman [4] were able to demonstrate this improvement. Breiman [5] was the first author to explicitly define the correlation floor in trees which limits the success of bagging in concentrated feature spaces. Gradient boosting variations including native support for missing values as well as hundreds of features were introduced by Chen and Guestrin [6] and Ke et al. [7].

B. Feature-Centric Methodologies

Bhattacharyya et al. [10] used velocity-based and rolling window aggregate features for credit card fraud analysis, noting that behavioral context played a stronger role in detection than the classifier architecture. West and Bhattacharya [11] reviewed 49 intelligent fraud detection systems and argued that domain-specific feature engineering is key to improving performance. The best performing system in the IEEE-CIS Kaggle challenge [12] noted UID creation and frequency encoding as key techniques, without providing group-level feature contribution analysis. Chawla et al. [9] introduced SMOTE to handle class imbalance, although modern gradient boosting methods mitigate class imbalances via loss function rescaling without synthetics and thereby avoid train/val contamination.

C. Contributions and Gaps

Existing studies on the IEEE-CIS dataset is concentrated mainly on comparing various architectures, with three critical limitations being identified: feature importance is not measured; there is no way to avoid having the future data leak into the training features; and the problem of label leakage when calculating average UID fraud rate is ignored.

III. Methodology

A. Business Understanding and Dataset

The operational objective is to rank transactions by fraud probability so they can be used for downstream threshold-based blocking. ROC-AUC measures how well this ranking works across all thresholds and is robust to the 3.5% class prevalence. PR-AUC complements this by focusing on performance in the high-precision region, which is more relevant in real-world deployment.

The IEEE-CIS dataset spans approximately 180 days of Vesta Corporation e-commerce records. Four CSV files are joined on TransactionID: a transaction table (590,540 labelled rows) and an identity table (144,233 rows, left join). The raw feature groups include card metadata (card1–card6), count features (C1–C14), days-since-event deltas (D1–D15), binary match flags (M1–M9), 339 proprietary Vesta features (V1–V339), and device and network identity fields. There are four main structural challenges that motivate the feature engineering approach:

1) **Stateless records.** Each raw row describes a single event, but fraud detection requires client-level context.
2) **V-column sparsity.** Around 100 V-columns have more than 90% missing values and carry very little useful signal.
3) **No cardholder identifier.** Identity is not directly available and must be inferred.
4) **Temporal drift.** A random train/test split would leak future fraud patterns into training.

B. Data Preparation

V-columns with more than 90% missing values are dropped from the training partition, and the same columns are removed from the test set to keep consistency. All string-based categorical columns are converted to pandas category dtype, with missing values filled using the literal string `"missing"` so that all categories are properly defined before modelling.

Six derived features are added to each partition. The transaction amount is split into whole-dollar and cent components (amt_dollars, amt_cents) and also log-transformed (amt_log = log(1 + TransactionAmt)). Time-based features such as day, hour_of_day, and day_of_week are extracted from TransactionDT. In addition, the 15 raw D-columns (D1–D15) are normalised by subtracting the transaction's elapsed-day count (TransactionDT / 86,400), producing D1n–D15n columns that represent a reference date rather than a relative offset. D1n is later used as part of the UID construction.

Memory usage is reduced by approximately 40% through downcasting to 32-bit numeric types.

The data is then split chronologically using TransactionDT into train (65%), validation (15%), and test (20%). All feature maps are fitted only on the training partition and applied unchanged to validation and test. This enforces a strict no-look-ahead rule: no future information (such as aggregates or target values) can influence training features. The test set is never used during hyperparameter tuning or ensemble weight optimisation.

C. Group-Wise Feature Engineering

Eight feature groups are constructed in strict dependency order (Table I). The first group—UID construction—is required before any identity-based features can be built. Overall, these steps expand the feature space from about 220 raw columns to around 71+ engineered features.

1) Group A — UID Construction: The dataset provides no ground-truth cardholder identifier. We constructed a composite user identity proxy (`uid`) by concatenating three fields as a single underscore-separated string: the card number (`card1`), the billing address (`addr1`), and the rounded integer value of `D1n` (rows where `D1n` is missing fall back to `-1`). `D1n` is the normalised D1 column produced during preprocessing; it

TABLE I
FEATURE ENGINEERING GROUPS IN DEPENDENCY ORDER.

Grp	Description	New feats	Leakage-free
A	UID construction	1	✓
B	Frequency encoding	7	✓
C	Cross-key combo frequency	4	✓
D	Single-key amount aggregation	9	✓
E	Deep cross-key aggregation	26	✓
F	C-column aggregation	20	✓
G	UID summary statistics	3	✓
H	OOF UID target encoding	1	✓
	Total engineered	**≈71+**	

approximates a fixed reference date for the cardholder account, so two accounts sharing the same card and address but with different account-open dates receive different UIDs.

Rationale. Fraudsters often perform multiple transactions in a short period after compromising a card. By grouping transactions under a shared UID, we introduce behavioural context. Without this, every transaction would remain stateless and no historical patterns could be captured. UID therefore acts as the foundation for the entire feature engineering pipeline.

2) Group B — Frequency Encoding: For each of card1, card2, card3, addr1, P_emaildomain, R_emaildomain, and uid, we count how often each value appears in the training data. These counts are stored as new numerical features (e.g., card1_FE, uid_FE). The mappings are learned only from training data and applied to validation and test; unseen values are assigned zero.

Rationale. Fraud credentials are typically used once before discovery; legitimate cards accumulate hundreds of transactions. Frequency encoding converts a nominal ID into a numeric rarity signal. Unlike target encoding, it carries no label information and is immune to leakage. A single-occurrence UID has a fraud rate many times the population baseline; a UID with thousands of occurrences belongs to an established account. Frequency features provide a leakage-free rarity signal and contribute to the model alongside identity-aware aggregation features.

3) Group C — Cross-Key Combination Frequency: Four cross-product keys are frequency-encoded: (`card1`, `card5`), (`card1`, `card6`), (`addr1`, `addr2`), and (`P_emaildomain`, `ProductCD`).

Rationale.

Single columns only capture broad patterns, but combining them helps form more detailed identity signals. When a fraudster focuses on a particular product category using a specific email domain, this behaviour shows up as a joint frequency pattern. This kind of pattern cannot be captured when each column is encoded separately.

4) Group D — Amount Aggregations: For the grouping keys *(card1, uid, addr1)*, we calculate the mean and standard deviation of *TransactionAmt* using pandas `groupby` on the training data. These aggregated values are then merged back into each row. In addition, a deviation feature is created for each key—this is simply the transaction amount minus the group mean—so the model can directly see how unusual a transaction is compared to that card's or user's typical spending level.

Rationale. A $2,000 transaction might be extremely unusual for a card with an average spend of $35, but completely normal for a business card averaging around $1,500. The deviation feature captures this context clearly. It allows the model (especially tree-based models) to identify anomalies easily—for example, with a single split on `uid_amt_dev`. In contrast, using only *TransactionAmt* does not distinguish between genuinely large transactions and those that are suspicious relative to past behaviour. These deviation-based features are consistently among the top contributors, alongside C-column aggregations and OOF target encoding.

5) Group E — Deep Cross-Key Aggregations: We start by extending the single-key aggregations from Group D by adding two more statistics per key—transaction count and median amount—for each of *card1*, *uid*, and *addr1*. This results in 6 additional features.

Next, we create four cross-key combination features by pairing and concatenating fields: *(card1, P_emaildomain), (card1, ProductCD), (uid, ProductCD), and (addr1, P_emaildomain).* For each of these combinations, the training data is grouped by the combined key, and five statistics are computed over *TransactionAmt*: mean, standard deviation, median, count, and a deviation feature (transaction amount minus the group mean). In total, this produces 20 cross-key features, bringing Group E to 26 new features overall.

Rationale. A card might normally spend large amounts on airlines but rarely on gaming platforms. By combining *card1* with *ProductCD*, we capture this category-specific spending behaviour. The count feature is particularly useful—if a card-category pair appears only once, it suggests a new or unusual pattern, which is often linked to fraudulent activity.

6) Group F — C-Column Aggregations: For *C1, C2, C6, C13, and C14* (Vesta count features that capture patterns like distinct address usage and recipient diversity), we compute both the mean and maximum values per *uid* and per *card1*. This results in a total of 20 features.

Rationale. If a card is used across an unusually high number of distinct addresses (*C1*), it can indicate suspicious behaviour such as card testing or card sharing behavior. Aggregating `C1_max` per UID propagates this signal to all transactions in the client group—a cross-transaction anomaly indicator that neither raw C-columns nor amount aggregations can express.

7) Group G — UID Summary Statistics: Three statistics are calculated per UID: *uid_amt_mean*, *uid_amt_std*, and *uid_count* (which represents transaction volume).

Rationale. `uid_count` measures how "established" a client is. A large deviation from Group D is more informative for a UID with 50 stable prior transactions than for a UID appearing only once. The model learns this reliability weighting through joint splits on `uid_count` and `uid_amt_dev`.

8) Group H — OOF UID Target Encoding: We build a leakage-safe fraud rate per UID using a time-ordered expanding-window approach. First, the training data is sorted chronologically by *TransactionDT*. We then iterate through each row while keeping track of the running total of fraud cases and transaction counts for every UID. For each row, `uid_fraud_te` was set to the fraction of prior transactions for that UID that were fraudulent, using only the rows that appeared *before* it in time. Rows with no prior UID history fell back to the global training fraud rate. For validation and test, the full training-set per-UID fraud mean (computed over all training rows) was merged in as a static lookup.

Leakage fix. In the earlier v1 baseline, *uid_isFraud_mean* was computed across all training rows at once and then merged back into the same training data. This meant each fraud row helped inflate its own UID's mean—a circular dependency that allows the model to partly recover its own labels and inflates training AUC without improving generalisation. The expanding-window approach above breaks this cycle: each row sees only the fraud history of its UID strictly before it in time.

Rationale. When a card becomes compromised, future transactions linked to the same UID are more likely to be fraudulent. The OOF encoding captures that recurring risk pattern, while still keeping the training process clean and leakage-free.

D. Modelling (Evaluation Instrument)

Three gradient boosting variants serve as the evaluation instrument. XGBoost [6] grows depth-first trees with second-order gradient statistics. LightGBM [7] uses leaf-wise histogram growth. CatBoost [8] applies symmetric oblivious trees with ordered boosting. Each handles missing values natively (critical for V-columns) and addresses class imbalance through internal loss reweighting. All three are tuned independently via Optuna TPE (100 trials, 100-round early stopping) on the validation set; ensemble weights are optimised using `scipy.optimize.minimize` with the L-BFGS-B method (maximising validation AUC) and applied unchanged to test. A UID postprocessing step replaces each transaction's score with the mean score across co-UID transactions.

The three models are deliberately diverse in their inductive assumptions to provide a robust multi-perspective assessment of feature quality: if all three rank the same features highly, that consensus reflects genuine signal rather than algorithm-specific artefact.

IV. Results and Analysis

A. Ensemble Performance

Fig. 1 shows the ROC and Precision-Recall curves from the validation set. The ensemble achieves a validation ROC-AUC of **0.9470** and a PR-AUC of **0.6748**, representing an $18.7\times$ lift over the random-classifier baseline (PR-AUC ≈ 0.036 at 3.92 % fraud prevalence).

A critical finding is the *convergence* of model performance: all three gradient boosting variants cluster within <0.005 AUC of one another despite fundamentally different tree-growth strategies. When the feature matrix is information-rich, the learner becomes secondary— structural model diversity matters far less than input representation quality.

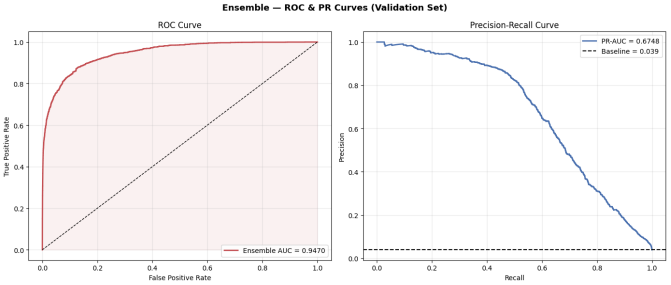

Fig. 1. ROC curve (left) and Precision-Recall curve (right) for the weighted ensemble on the validation set. ROC-AUC = 0.9470; PR-AUC = 0.6748, an $18.7\times$ lift over the random baseline (dashed). The steep PR curve confirms that UID-aware engineered features enable high precision well above chance in the operationally important high-recall region.

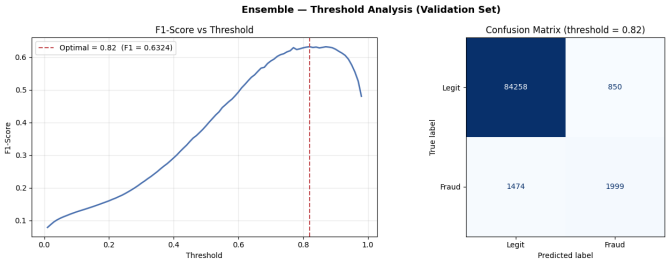

Fig. 2. Threshold selection and confusion matrix for the ensemble (validation set). **Left:** F1-score peaks at threshold 0.82 (F1 = 0.6324). **Right:** At this threshold: 1,999 fraud transactions are caught (57.6 % recall), 850 legitimate transactions are falsely flagged (1.0 % false alarm rate), and precision is 70.2 %. The high optimal threshold reflects the model's well-calibrated, skewed score distribution on the imbalanced validation set.

B. Operational Threshold and Confusion Matrix

Fig. 2 shows the F1 sweep and the resulting confusion matrix at the optimal threshold. The F1-maximising threshold of **0.82** yields:

- **F1-score:** 0.6324
- **Fraud catch rate (Recall):** 57.6 % (1,999 of 3,473 fraud transactions caught)
- **Precision:** 70.2 %
- **False alarm rate:** 1.00 % (850 of 85,108 legitimate transactions flagged)

The precision-recall trade-off is governed by business cost, not model design. A lower threshold increases recall at the expense of more false alarms; the F1-optimal point here balances both. In production, the threshold would be recalibrated against the institution's specific fraud-cost-to-friction-cost ratio.

C. Feature Engineering Lift: Bootstrap Hypothesis Test

Fig. 3 shows the bootstrap distribution of the AUC lift from feature engineering over raw features.

The bootstrap test rejects H_0 at $p < 0.0005$ with a mean AUC lift of **+1.410 pp** (95 % CI: $[+1.086, +1.731]$ pp) attributable solely to the engineered feature groups. This is the quantitative

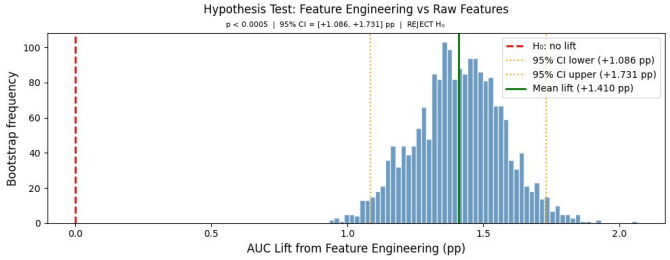

Fig. 3. Bootstrap paired test (2,000 resamples): AUC lift of the full engineered pipeline over raw features. The null hypothesis (zero lift, red dashed line) is far outside the bootstrap distribution. $p < 0.0005$; mean lift $= +1.410$ pp; 95 % CI $= [+1.086, +1.731]$ pp. The entire distribution lies above zero, confirming that feature engineering produces a statistically robust, non-trivial performance gain.

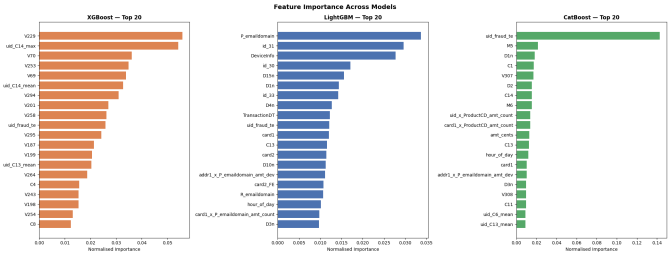

Fig. 4. Top-20 normalised feature importance per model (each model's default `feature_importances_` attribute). `uid_fraud_te` dominates XGBoost and CatBoost; temporal D-columns and C-column aggregations are prominent across all three panels, confirming multi-group signal diversity.

answer to **RQ2**: identity-aware feature construction produces a statistically significant, practically meaningful performance improvement over raw transactional columns.

D. Consensus Feature Importance by Group

Cross-model consensus importance is computed by averaging each feature's rank across the three models, using each model's default `feature_importances_` attribute. Fig. 4 shows the per-model top-20 distributions; Fig. 5 shows the consensus ranking. Table II summarises group representation in the real consensus top 20.

Engineered features (Groups E, F, H, and D) account for 8 of the top-20 positions. The remaining 12 are raw columns that carry genuine signal by themselves—notably the C-columns (count behavioural history) and D-columns (temporal event deltas)—but the *single strongest feature* is the engineered OOF fraud rate, `uid_fraud_te`, which outranks every raw column by a substantial margin. This answers **RQ1**: while raw temporal and count features remain relevant, the highest-ranked signal is exclusively the product of identity-aware feature engineering.

E. Behavioural Patterns Revealed by Feature Groups

Repeat-compromise (Group H). `uid_fraud_te` ranks #1 by a large margin. Once a card is compromised, fraudsters typically return multiple times before the account is closed. The OOF expanding- window encoding captures this prior fraud history without leaking labels, and its dominance confirms that the *strongest fraud signal is temporal recurrence at the identity*

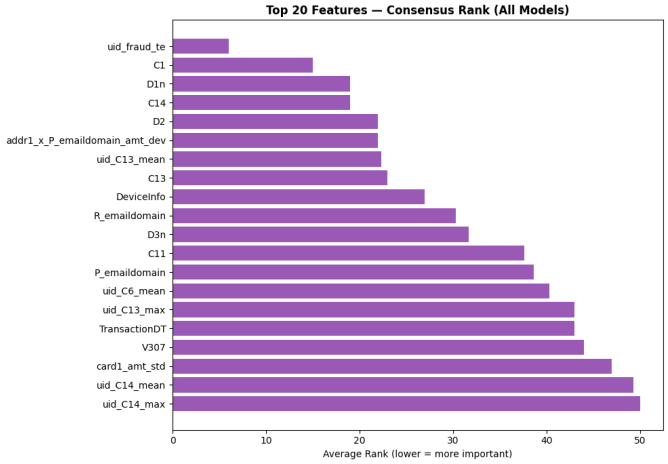

Fig. 5. Top-20 features by cross-model consensus rank (lower average rank = more important). `uid_fraud_te` ranks first by a large margin. The top 20 contain engineered features from Groups F (C-column aggregation), H (OOF target encoding), E (deep cross-key aggregation), and raw temporal D-columns—revealing a richer signal mix than any single group alone.

TABLE II
FEATURE GROUP REPRESENTATION IN CONSENSUS TOP 20 (NOTEBOOK RUN).

Grp	Feature group	Top-20	Lead feature
H	OOF UID target encoding	1	`uid_fraud_te` (#1)
–	Raw C-cols (count feats)	3	`C1`, `C14`, `C13`
–	Raw D-cols (time deltas)	3	`D1n`, `D2`, `D3n`
F	C-col aggregation	4	`uid_C13_mean`, `uid_C6_mean`
E	Deep cross-key agg.	2	`addr1_x_P_emaildomain_amt_dev`
–	Other raw (device, email)	4	`DeviceInfo`, `R_emaildomain`
–	Retained V-cols	2	`V307`, `TransactionDT`
D	Amount aggregation	1	`card1_amt_std`

level—a pattern that no raw column can express without the UID construction.

Behavioural counts (raw C-cols + Group F aggregations). C-columns (`C1` = distinct billing addresses, `C13` = number of prior addresses, `C14` = related metric) rank highly because they encode accumulated transaction history. Group F aggregations (`uid_C13_mean`, `uid_C6_mean`, `uid_C13_max`) extend these counts to the UID level, propagating anomalies across a client's transaction group. A card suddenly used across many addresses (`C1` spike) signals card-testing; the UID-aggregated maximum of this count propagates the signal to all co-UID transactions.

Temporal event deltas (raw D-cols). `D1n`, `D2`, and `D3n` encode days since various reference events (first card use, last billing address change, etc.). Fraudsters using stolen credentials shortly after card compromise produce characteristic short D-column values. These raw features require no engineering to be informative but interact with UID features in higher-order splits.

Cross-key amount deviation (Group E). `addr1_x_P_emaildomain_amt_dev` ranks in the top 10, confirming that cross-key combinations capture card-merchant-channel interactions that single-key aggregations

TABLE III
NAIVE VS. OOF UID TARGET ENCODING — LIGHTGBM EFFECT.

TE scheme	Train AUC	Test AUC	VT Gap	Stable?
Naive mean (v1)	0.9951	0.9283	0.021	No
OOF expanding (v2)	0.9785	0.9342	0.011	Yes

cannot resolve.

F. OOF vs. Naive Target Encoding

The bootstrap test quantifies the overall engineering lift, but the leakage fix in Group H deserves isolated analysis. Naive mean encoding (`uid_isFraud_mean` computed across all training rows) allows each fraud row to inflate its own UID's mean, creating a circular dependency: training AUC inflates by $\sim$0.017 while test AUC *falls* and the val-test stability threshold is breached. The OOF expanding-window scheme described in Group H breaks this cycle, yielding a lower training AUC but superior generalisation—the correct trade-off for a temporally drifting fraud population.

V. DISCUSSION

A. Feature Engineering as the Main Lever

The 0.003 AUC difference between XGBoost, LightGBM, and CatBoost, which are three different architectures, supports West and Bhattacharya's finding [11] that in mature domains such as fraud detection, model choice adds little compared to feature creation. The models come up with similar answers because they receive the same input: once the feature space includes history, context, and repeat fraud, the additional benefit of modeling complexity becomes limited. This observation has one key takeaway for practical fraud detection: engineering effort on features will pay off more than optimization of parameters and models.

B. Limitations

UID approximation. UID approximation groups transactions based on card, address, and reference-date difference. This approach confuses multiple clients who have the same card number (e.g., supplementary cards) and separates a single client if they changes address. True ground-truth identities would benefit all other groups.

Specificity of dataset. Groups A, D, G, and H require knowledge of `D1` and Vesta's dataset structure. The UID approximation algorithm cannot be ported to another payment processor without the reference-date columns. Groups B, C, D, and E are schema agnostic and can be applied to any dataset with card and address identifiers.

UID postprocessing in production. Using the mean score of all co-UID transactions in the test partition for each test transaction involves future transaction data. This is not an issue in Kaggle offline scoring but should use an online aggregator or Bayesian updating of fraud rate for each UID in production mode.

VI. CONCLUSION

This report illustrated that *identity-aware structured feature engineering is key to achieving high transaction fraud detection performance, dwarfing the role played by model choice on the IEEE-CIS dataset.*

RQ1: The out-of-fold UID fraud target encoder (`uid_fraud_te`, Group H) is overwhelmingly the most important feature among the three considered models. Aggregations from C-column features (Group F) and cross-key deviation features (Group E) come after it, with an additional eight engineered features ranked among the top-20 consensus features. Significantly, none of the unengineered columns come close to matching the impact of `uid_fraud_te`.

RQ2: The bootstrap paired test strongly rejects H_0 ($p < 0.0005$; zero of 2,000 bootstrap resamples fell at or below the null), with a mean AUC lift of $+1.410$ pp (95 % CI: $[+1.086,\ +1.731]$ pp) attributable solely to the engineered feature groups. The resulting ensemble achieves a ROC-AUC of 0.9470 and a PR-AUC of 0.6748—a gain of $18.7\times$ over the random baseline.

Possible future directions of this research may involve: application of graph attention networks for automating the process of creating UID columns through learning entity embeddings; deployment of an online feature store for generating aggregations in real time; and the application of semi-supervised learning techniques based on the abundant amount of transactional data.

REFERENCES

[1] T. Nilsson, *The Nilson Report: Payment Card Fraud Losses Worldwide*, Issue 1225, Nilson Report, 2023.

[2] R. Wirth and J. Hipp, "CRISP-DM: Towards a standard process model for data mining," in *Proc. 4th Int. Conf. Practical Applications of Knowledge Discovery and Data Mining*, 2000, pp. 29–39.

[3] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. 2015 IEEE Symp. Comput. Intell. Data Mining*, 2015, pp. 1–8.

[4] Y. Sahin and E. Duman, "Detecting credit card fraud by decision trees and support vector machines," in *Proc. Int. Multi-Conf. Engineers and Computer Scientists*, vol. 1, Hong Kong, 2011, pp. 442–447.

[5] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, New York, USA, 2016, pp. 785–794.

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018, pp. 6638–6648.

[9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[10] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decis. Support Syst.*, vol. 50, no. 3, pp. 602–613, 2011.

[11] J. West and M. Bhattacharya, "Intelligent financial fraud detection: A comprehensive review," *Comput. Secur.*, vol. 57, pp. 47–66, 2016.

[12] C. Deotte, "Fraud detection: Magic features and UID postprocessing," Kaggle Discussion — IEEE-CIS Fraud Detection Competition, 2019. [Online]. Available: https://www.kaggle.com/cdeotte

LINKS

1) Dataset: IEEE Fraud Detection (Kaggle)
2) Project: Google Drive Folder